

Trabajo Final

Computación Evolutiva



Problema del Viajante Asimétrico (ATSP)



Integrantes

González Tesoro, Joaquín. jgonzaleztesoro@alumnos.exa.unicen.edu.ar.

Roumec, Iñaki. iroumec@alumnos.exa.unicen.edu.ar.

Índice

Introducción.....	3
Diseño de la Aplicación.....	3
Lenguaje de Programación.....	4
Configuración.....	4
Ejecución.....	4
Parámetros.....	5
Ejecución.....	5
Condiciones de Corte.....	5
Selección.....	6
Cruce.....	6
Mutación.....	6
Mejora o Búsqueda Local.....	6
Selección de Sobrevivientes o Proceso de Inserción.....	6
Operadores.....	7
Selección.....	7
Tournament.....	7
Roulette.....	7
Cruce.....	7
OX1.....	7
PMX (Partially Mapped Crossover).....	8
Mutación.....	9
Swap.....	9
Scramble.....	9
Inversion.....	9
Mejora o Búsqueda Local.....	9
Selección de Sobrevivientes o Proceso de Reinserción.....	10
Experimentación.....	11
Lectura de Gráficos.....	11
Experimentos.....	12
Primer Experimento.....	13
Segundo Experimento.....	14
Tercer Experimento.....	15
Cuarto Experimento.....	17
Quinto Experimento.....	18
Mejor Configuración.....	19
Problemas Hallados.....	20
Endogamia.....	20
NumPy y Numba.....	20
Conclusiones.....	22

Introducción

En este trabajo, a partir del análisis de operadores y valores realizado en el anterior informe, se realiza la descripción y análisis de resultados de la aplicación desarrollada para resolver el **Problema del Viajante** o **Travelling Salesman Problem (TSP)**. Este es un problema de optimización **NP-Hard**, lo que significa que no se conoce un algoritmo de tiempo polinomial para resolverlo exactamente, y representa a diversos problemas de optimización que ocurren en una amplia variedad de dominios reales de aplicación. A continuación, se enuncia el problema:

Dado un conjunto de N ciudades y el costo del viaje entre cada par de esas ciudades, encontrar el camino de mínimo costo que permite visitar todas las ciudades del conjunto (pasando solamente una vez por cada ciudad) y retornar a la ciudad de partida.

En particular, existen dos variantes para este problema: la variante asimétrica (el costo de ir de la ciudad i a la ciudad j puede no ser el mismo que el de ir de la ciudad j a la ciudad i), abreviada como **ATSP**; y la variante simétrica (el costo de ir de la ciudad i a la ciudad j es el mismo que el de ir de la ciudad j a la ciudad i), también llamada **Problema del Viajante de Steiner** y abreviada **STSP**. La aplicación diseñada se centra en la primera variante: la variante ATSP.

Diseño de la Aplicación

La aplicación diseñada permite resolver el problema del viajante ATSP haciendo uso de **algoritmos meméticos o memetic algorithms (MA/GLS)**, los cuales combinan **algoritmos evolutivos o evolutionary algorithms (EA)** con búsqueda local intensiva. Este enfoque ha demostrado ser altamente efectivo en problemas de optimización combinatoria, como lo es el problema del viajante.

La aplicación adopta un **modelo steady-state (Estado Estacionario)**, el cual consiste en la “renovación continua o incremental”. En cada iteración, los hijos, luego de haber sido mutados y haberles aplicado una búsqueda local, son insertados en la población actual, de acuerdo al proceso de selección de sobrevivientes seleccionado.

Las soluciones son representadas o codificadas como permutaciones. En esta representación, la adyacencia conforma una restricción fundamental, puesto que dictamina el orden en el que las ciudades son recorridas. Es decir, **no importa el lugar en el que una ciudad aparezca, sino junto a qué ciudades aparece**. Esta es la representación estándar del problema y se la denomina *path representation*. En esta, la primera ciudad se conecta implícitamente con la última. Esto implica que **[3, 2, 0, 1]** es equivalente a **[0, 1, 3, 2]**, ya que proponen el mismo recorrido. Si bien son equivalentes, ambas permanecen en la población, ya que, debido a que no representan la misma permutación, la aplicación de operadores pueden llevarlas a lugares distintos del espacio de búsqueda, aumentando la exploración del algoritmo. La utilización de esta representación es válida puesto que se trabaja con circuitos cerrados (siempre se vuelve a la ciudad de partida).

Siguiendo la perspectiva evolutiva de la “supervivencia del más apto”, se considera al problema como un problema de maximización. Con esto en cuenta, se define al *fitness* como: $1/L$, donde L es la distancia recorrida al realizar el circuito. De esta forma, **caminos más cortos, resultan en un mayor fitness y un mayor fitness resulta en una mejor solución al problema**.

Considerando todo lo anterior, se describe el lenguaje de programación utilizado para el desarrollo de la aplicación, cuáles y cómo puede realizarse la configuración de sus parámetros y cómo ejecutarla.

Lenguaje de Programación

La aplicación decidió construirse en Python. Si bien este no es un lenguaje eficiente, presenta una sintaxis y legibilidad elevada. Por consiguiente, a fines académicos, se consideró la mejor opción, ya que facilita la legibilidad de los algoritmos.

Para compensar la pérdida de eficiencia, la aplicación permite utilizar multiprocesamiento para ejecutar en paralelo las distintas ejecuciones de una misma configuración. Adicionalmente, utiliza *caching* en la carga de la matriz, lo que reduce el tiempo de acceso a disco y, por consiguiente, el tiempo de ejecución total.

Configuración

La configuración del algoritmo se realiza a través de un archivo YAML, aplicando la práctica de *configuration as code* (AaC). YAML es un formato de serialización de datos legible. A través, en un único archivo, es posible configurar todos los parámetros de la aplicación, facilitando su reproducción de ejecución y legibilidad.

Otras alternativas que se evaluaron para la configuración del algoritmo fueron la presentación de una interfaz gráfica y la posibilidad de pasar los parámetros como comandos de la ejecución del archivo. No obstante estas fueron descartadas. La interfaz gráfica fue descartada debido a que dificulta la reproducibilidad de las ejecuciones, haciendo el proceso más tedioso. Por otro lado, el uso de la línea de comandos fue descartada debido al extenso número de parámetros que la aplicación requiere.

Dentro de la carpeta del proyecto **resources** es posible hallar un archivo **configuration.example.yml**. Este presenta un ejemplo de configuración de la aplicación. **Para configurar sus propios parámetros, solo debe copiar el archivo y cambiar los valores de los campos.** Para cada uno de estos, se especifica, mediante comentarios, las opciones disponibles.

Ejecución

Para ejecutar la aplicación, debe ejecutar el siguiente comando, reemplazando el archivo de configuración de ejemplo propuesto por la ruta relativa o absoluta del archivo de configuración (o conjunto de estos) que desea ejecutar.

```
make run CONFIG="resources/configuration.example.yml"
```

Si no cuenta con Make, puede ejecutar la aplicación activando el entorno de Python, mediante el comando **"source venv/bin/activate"** (Linux) ó **".\venv\Scripts\activate.bat"** (Windows). Luego, ejecute el siguiente comando, reemplazando la ruta provista como ejemplo por su ruta de interés:

```
python3 -u src/main.py --config "resources/configuration.example.yml"
```

Una vez ejecutada la aplicación, es posible observar en la terminal donde se ejecuta el comando un seguimiento del proceso de ejecución o, en caso de que la configuración contenga errores, los mensajes de error correspondientes.

Parámetros

A continuación, se proporciona una explicación de los distintos parámetros que pueden hallarse en el archivo de configuración.

Ejecución

Estos parámetros se conforman por:

- **instance**: Instancia del problema a resolver.
- **executions**: Número de veces que se ejecutará cada configuración.
- **population_size**: Tamaño de la población, es decir, su cantidad de individuos.
- **percentage_of_random_generated_individuals**: La aplicación permite la posibilidad de realizar una **inicialización híbrida** o **mixta**, complementando la inicialización aleatoria con heurísticas. En este parámetro, es posible definir el porcentaje de individuos de la población generados de forma aleatoria. Se recomienda, en base a lo discutido en el anterior informe, un valor de 0.8. El resto de la población se genera utilizando una heurística, en particular, la heurística del **vecino más cercano** o **nearest neighbour (NN)**. Esta consiste en construir recorridos agregando siempre la ciudad más cercana no visitada. Un valor de 1.0 inhabilita el uso de esta heurística; no obstante, esto no se recomienda, puesto que la inicialización híbrida ha demostrado presentar muy buenos resultados, tal y como se detalló en el anterior informe.
- **multiprocessing**: De activarse esta opción, la aplicación utilizará multiprocesamiento para ejecutar las distintas ejecuciones de una configuración en paralelo. Se recomienda activarla únicamente si el número de ejecuciones es un valor alto (mayor a 20), ya que, de otra forma, produce un *overhead* que podría perjudicar el rendimiento en lugar de mejorarlo.

Condiciones de Corte

Sus parámetros se conforman por:

- **generations**: Si se activa, el algoritmo se detendrá al alcanzar cierto número de generaciones.
- **max_generations**: Si la opción anterior está activa, la ejecución de una configuración se detendrá al alcanzar el número de generaciones indicadas en este campo.
- **generations_without_improvements**: Si se activa, el algoritmo se detendrá al alcanzar un cierto número de generaciones consecutivas sin mejoras. Se recomienda mantener esta opción desactivada, puesto que definir el criterio de convergencia (número de generaciones sin mejora) de forma acertada suele ser difícil cuando no se tiene experiencia previa en el uso de este tipo de algoritmos. Si el criterio no es definido de forma acertada, se puede detener el algoritmo prematuramente y evitar que alcance la mejor solución que podría alcanzar.
- **max_generations_without_improvements**: Si la opción anterior está activa, la ejecución de una configuración se detendrá al alcanzar el número de generaciones consecutivas sin mejora indicadas en este campo.

Al menos uno de los dos criterios de detenimiento (***generations*** o ***generations_without_improvements***) debe estar activo para que la aplicación funcione. En caso de no cumplirse esto, la aplicación levantará una excepción. Esto se debe a que, si el algoritmo no cuenta con una condición de corte, este se ejecutará indefinidamente.

Selección

A continuación, se describen los parámetros configurables con respecto al proceso de selección de padres, es decir, el proceso de selección de aquellos individuos que pueden generar hijos:

- ***operator***: Operador de selección a utilizar. Los operadores disponibles son **torneo** o ***tournament*** y **ruleta** o ***roulette***. Como se menciona en el informe anterior, no hay rechazo generalizado de ninguno, pues, ambos han demostrado ser igualmente buenos.
- ***selected_individuals***: Número de individuos de la población seleccionados para participar en la selección de padres.
- ***tournament_size***: En caso de que el operador seleccionado sea ***tournament***, se debe especificar su parámetro k . En otro caso, el valor de este campo será ignorado.

Cruce

Los parámetros configurables con respecto al proceso de crossover, es decir, el proceso de cruce de padres para generar hijos son los siguientes:

- ***operator***: Operador de cruce a utilizar. Los operadores disponibles son **OX1** y **PMX**. **OX1** se enfoca en preservar el orden relativo de las ciudades al rellenar espacios vacíos. **PMX** copia un segmento del padre y hace mapeos para construir la descendencia.
- ***probability***: Probabilidad necesaria para que dos padres seleccionados se crucen.

Mutación

A continuación se describen los parámetros configurables del proceso de mutación, utilizado para introducir variabilidad en los individuos generados en la etapa de cruce.

- ***operator***: Los operadores de mutación implementados son **Inversion**, **Scramble** y **Swap**. En secciones posteriores del informe se realiza la explicación de cada uno de ellos.
- ***probability***: Probabilidad de que un individuo sea modificado por mutación. Suele definirse de forma inversamente proporcional al tamaño de la población, o bien se recomienda utilizar valores bajos (por ejemplo, entre 0.001 y 0.1).

Mejora o Búsqueda Local

Para mejorar los individuos generados tras el cruce y la mutación, se aplica un algoritmo de búsqueda local que lleva a cada individuo a un óptimo local antes de reinsertarlo en la población. Este paso se enfoca en el refinamiento de las soluciones obtenidas.

- ***operator***: El operador utilizado para realizar la búsqueda local es el **insertion** o **inserción**, cuyo funcionamiento se detalla más adelante.
- ***probability***: Probabilidad de que un individuo sea mejorado mediante búsqueda local.

Selección de Sobrevivientes o Proceso de Inserción

En esta etapa define cómo los individuos generados son insertados en la población y cuáles de los existentes sobreviven al proceso de reemplazo. Los parámetros configurables son:

- **operator**: Operador de reemplazo utilizado. En este caso, el único disponible es **replace worst**, que consiste en reemplazar los n peores individuos de la generación actual por los n mejores descendientes generados.
- **individuals_to_replace**: Número de individuos que se reemplazan en cada generación.

Operadores

A continuación, se provee una breve explicación de cada operador disponible en la aplicación.

Selección

El objetivo de estos operadores es seleccionar al conjunto de individuos que tendrán la posibilidad de generar hijos. La implementación de los operadores descritos a continuación puede hallarse en el siguiente directorio del proyecto: **src/operators/selection/algorithms**.

Tournament

Dada una población, selecciona a k individuos aleatoriamente. El individuo del subconjunto con mayor *fitness* queda seleccionado como apto para cruce. Este proceso se repite n veces, lo que resulta en n individuos seleccionados. Un individuo puede ser seleccionado más de una vez.

El parámetro k del algoritmo puede ser configurado en la aplicación utilizando el campo **tournament_size**, mientras que el parámetro n se configura utilizando el parámetro **selected_individuals**.

Roulette

Dada una población, designa a cada individuo una probabilidad de ser elegido en base a su *fitness*. A mayor es el *fitness* (y, por consiguiente, menor el costo del viaje y mejor la solución), mayor es la probabilidad. Luego, elige aleatoriamente, en base a dichas probabilidades, a n individuos.

El número de individuos seleccionados, n , puede ser configurado en la aplicación mediante el campo **selected_individuals**.

Cruce

El objetivo de estos operadores es cruzar a los individuos seleccionados mediante los operadores anteriores. El directorio con su implementación es: **src/operators/crossover/algorithms**.

OX1

Dados dos individuos, se seleccionan aleatoriamente dos índices de corte (|) que definen un segmento central. Los genes de uno de los individuos (**Padre 1**) dentro de dichos índices se copian directamente en el individuo hijo u **offspring**. El resto del cromosoma queda inicialmente vacío (_).

Padre 1: [3 | 5 2 1 | 4 6]

Offspring 1 Parcial: [_ | 5 2 1 | _ _]

Luego, se recorre el segundo individuo (**Padre 2**) desde el inicio y se coloca cada gen suyo que no exista ya en el *offspring* en los espacios vacíos, respetando el orden del recorrido.

Padre 2: [6 2 4 5 1 3]

(Los valores **2**, **5** y **1** se ignoran porque ya se hallan en el centro del *offspring*. Los valores restantes en orden son **6**, **4** y **3**).

Offspring 1 Resultante: [6 | 5 2 1 | 4 3]

(Para el **Offspring 2** se realiza exactamente el mismo procedimiento pero de manera inversa, copiando el centro del **Padre 2** y rellenando con los genes restantes del **Padre 1**).

PMX (*Partially Mapped Crossover*)

Dados dos individuos, se seleccionan aleatoriamente dos índices de corte (|) que definen un segmento central. El objetivo es heredar el segmento central de un padre y rellenar el resto manteniendo el orden y la estructura del otro padre mediante un mapeo de posiciones.

Padre 1: [3 5 | 2 1 | 4 6]

Padre 2: [6 2 | 4 5 | 1 3]

El segmento central del **Padre 2** se copia directamente en el **Offspring 1** en las mismas posiciones. El resto del cromosoma queda inicialmente vacío (_).

Offspring 1: [_ _ | 4 5 | _ _]

Seguido a eso, se definen las relaciones de mapeo exclusivas para el cruce en base en los elementos centrales de ambos padres: **2 se mapea con 4** y **1 se mapea con 5** (es decir, $2 \leftrightarrow 4$ y $1 \leftrightarrow 5$).

Luego, se intentan copiar los elementos externos del **Padre 1** en las mismas posiciones del **Offspring 1**. Si el elemento no está en el nuevo segmento central, se copia directamente:

- El **3** (posición 0 del Padre 1) no está en el segmento central [4, 5]. Se copia.
- El **6** (posición 5 del Padre 1) no está en el segmento central [4, 5]. Se copia.

Offspring 1 parcial: [3 _ | 4 5 | _ 6]

Los elementos restantes del **Padre 1** (**5** y **4**) estos sí generan duplicados porque ya existen en el segmento central del *offspring*. Para solucionar este inconveniente, se utilizan los mapeos definidos.

Offspring 1 Resultante: [3 1 | 4 5 | 2 6]

(Para el **Offspring 2** se realiza exactamente el mismo procedimiento pero de manera inversa, copiando el centro del **Padre 1** y rellenando con el mapa invertido desde el **Padre 2**).

Mutación

Los operadores de mutación se aplican sobre los *offspring* resultantes del proceso de cruce. El directorio con su implementación es: **`src/operators/mutation/algorithms`**.

Swap

Dado un individuo, se intercambian dos genes aleatoriamente. Ejemplo:

`[A B C D]`

Se intercambian los valores de los índices 0 y 2:

`[C B A D]`

Scramble

Dado un individuo, intercambia un segmento de sus genes, entre dos índices, aleatoriamente:

`[A B | C D E F | G H]`

Individuo resultante:

`[A B | F C E D | G H]`

Inversion

Dado un individuo, invierte un segmento aleatorio de sus genes:

`[A B | C D E F | G H]`

Individuo resultante:

`[A B | F E D C | G H]`

Mejora o Búsqueda Local

La búsqueda local es el proceso que, una vez generado un hijo, tras cruce y mutación, lleva, a través de la aplicación de un algoritmo rápido, a dicho individuo a su óptimo local antes de devolverlo a la población. **Mientras que el objetivo de la mutación es la exploración del espacio de soluciones, el objetivo de la búsqueda local es el perfeccionamiento de dichas soluciones.**

Como se hizo mención en el informe anterior, existen dos enfoques de búsqueda local: el enfoque **Lamarckiano**, donde después de optimizar localmente se reemplaza el genotipo por la versión mejorada, y el enfoque **Baldwiniano**, donde solo se actualiza la aptitud sin cambiar el genotipo original.

Debido a las razones mencionadas en el informe anterior, el enfoque por el que se optó aplicar es el **Lamarckiano**, ya que es el más común y efectivo en MA.

Con respecto al algoritmo seleccionado, se eligió el algoritmo de inserción o *insertion*. Este se encarga de tomar, en el contexto del problema, una ciudad y ubicarla en otra posición. Para entender esto mejor, tomar el siguiente cromosoma como referencia:

[<A B C D>]

El algoritmo elige uno de los genes (ciudades) y prueba insertarlo en diferentes ubicaciones. Por ejemplo, suponer que la ciudad elegida es B. El algoritmo probaría las siguientes combinaciones:

[<B A C D>, <A B C D>, <A C B D>, <A C D B>]

Finalmente, de dichas combinaciones, se elige la que presenta un mayor *fitness*. Esto, en el planteo de este trabajo, implica el menor costo de viaje. La razón por la que se elige la variante con el mayor *fitness* es debido a que, como se mencionó, la búsqueda local está orientada al perfeccionamiento de los individuos, llevándolo a un óptimo local antes de devolverlo a la población.

La implementación del algoritmo mencionada puede hallarse dentro de la carpeta del proyecto **src/operators/improvement/algorithms**.

Selección de Sobrevivientes o Proceso de Reinserción

Dado el esquema de ejecución elegido *Steady-State*, este proceso consiste en la evaluación de los individuos a reemplazar. Como se mencionó en el anterior informe, existen dos procesos posibles: *replace worst* y *tournament replacement*. Anteriormente, se había decidido optar por este último. No obstante, la elección final, por recomendación de la cátedra, fue el modelo *replace-worst*. Este consiste en reemplazar a las n peores soluciones de la población actual por las n mejores soluciones de las soluciones obtenidas luego de la secuencia selección de padres, cruce, mutación y búsqueda local. La razón detrás de la elección de este modelo es que, al usar esta variante, los algoritmos suelen funcionar de manera más eficiente.

La implementación del algoritmo correspondiente a este proceso puede hallarse en **src/operators/survivors/algorithms/replace_worst.py**. Esta consiste en, dada la población actual, ordenar sus miembros de acuerdo a su *fitness*. Posteriormente, tomar los m miembros con menor *fitness* y reemplazarlos por los m mejores individuos generados.

A considerar, se utilizó n en el primer párrafo y m en el segundo. Esto se debe a que no siempre es posible reemplazar a n individuos de la población, puesto que puede suceder que la cantidad de nuevos individuos sea menor a n ($m < n$). Esto es producto de la limitación impuesta al cruce entre individuos, la cual se detalla en la sección “Endogamia” del apartado “Problemas Hallados”.

Experimentación

A continuación, se describe el proceso de experimentación llevado a cabo: cada uno de los experimentos ejecutados y sus conclusiones. El objetivo de este proceso es hallar, dentro de un conjunto de parámetros variables, la mejor configuración para la instancia **p43**, brindada por la cátedra. El motivo por el que se optó por esta instancia y no por la instancia **b17** es debido a que, para esta última, independientemente de los parámetros a variar, siempre se llegaba a la solución óptima con mínima diferencia entre las pruebas.

El criterio de evaluación de las distintas configuraciones será el hallazgo de un costo menor. Es decir, se considerarán mejores aquellos parámetros que permitan, en general, hallar mejores valores (menores costos) para la instancia. En caso de empate, se considerarán otras cuestiones, tales como el tiempo promedio de ejecución.

Lectura de Gráficos

Los gráficos presentados en el informe se ven como el mostrado a continuación:

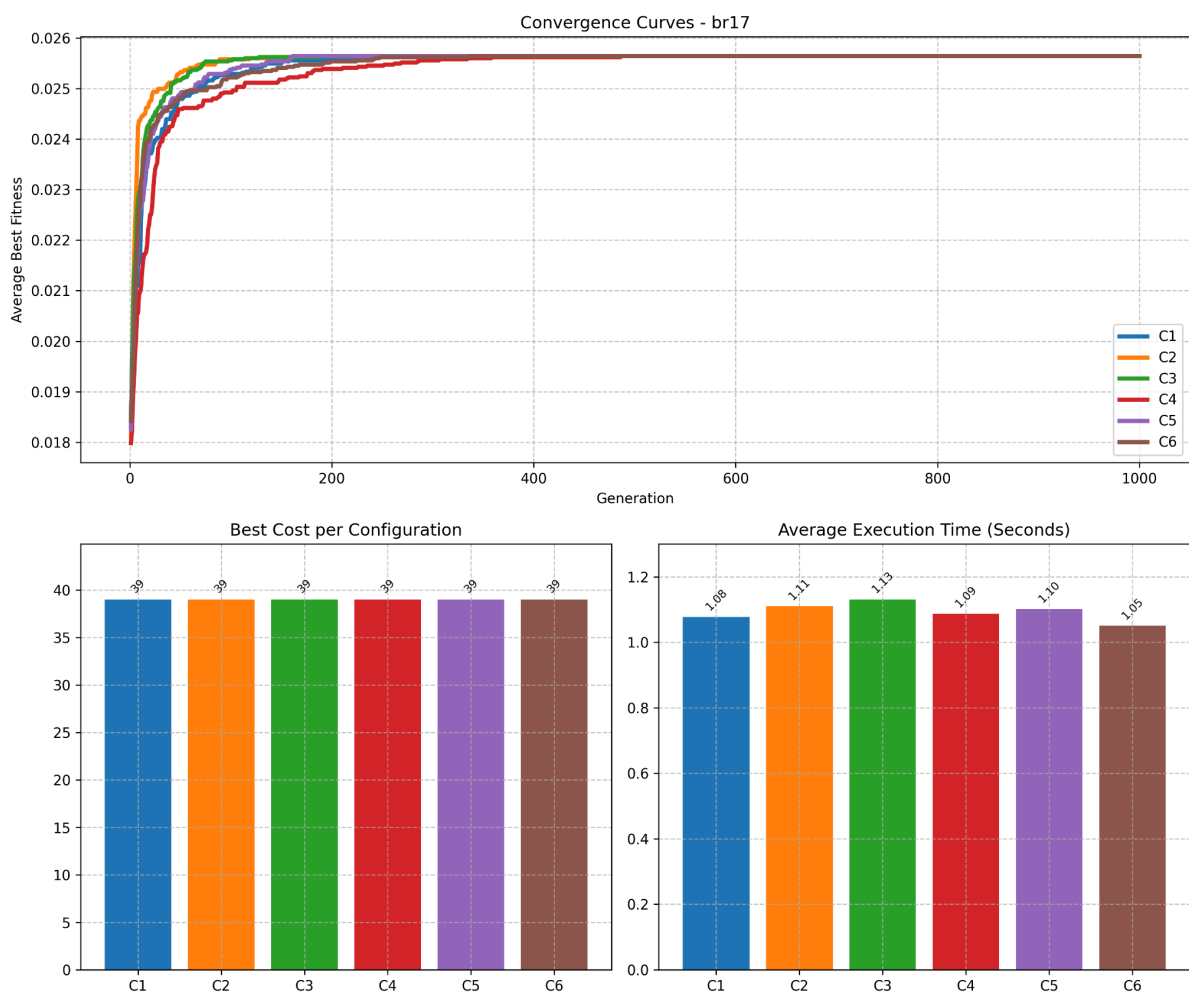


Imagen 1: Gráfico de ejemplo resultante de la ejecución de la aplicación.

En este, es posible observar tres subgráficos: en la parte superior, un gráfico de convergencia que diagrama el **fitness promedio por generación** de las distintas ejecuciones de la configuración objetivo; en la esquina inferior izquierda, un gráfico de barra que muestra **la mejor solución obtenida por la configuración**; y en la esquina inferior derecha, un gráfico de barras que muestra **el tiempo promedio de ejecución por configuración**.

Con motivos de legibilidad, en el informe se incluyen los gráficos sin los desvíos estándar. No obstante, en la carpeta del proyecto **outputs/**, pueden hallarse los resúmenes, gráficos combinados con y sin desvío estándar y gráficos individuales de cada uno de los experimentos.

Experimentos

A lo largo de los experimentos habrá una serie de parámetros fijos y un conjunto de parámetros variables. Es de este último conjunto que se busca determinar la mejor configuración. Estos parámetros que variarán entre los experimentos serán:

- **Ejecuciones por configuración:** {30, 80}, donde el objetivo de utilizar dos valores es una mayor precisión en las estadísticas obtenidas.
- **Operador de selección:** {tournament, roulette}.
- **Tamaño de torneo:** {3, 5, 10}.
- **Operador de cruce:** {OX1, PMX}.
- **Probabilidad de cruce:** {0.6, 0.9}.
- **Operador de mutación:** {swap, scramble, inversion}.
- **Probabilidad de mutación:** {0.1, 0.9}. La literatura recomienda valores bajos (0.1 o menores). No obstante, también resulta lógico pensar que una probabilidad alta de mutación podría llegar a dar buenos resultados, ya que permitiría escapar de óptimos locales con mayor facilidad. El objetivo de estos valores es confirmar que 0.1 es, en general, una mejor opción.

Para cada una de los parámetros anteriores, se busca determinar el mejor valor dentro del conjunto para la instancia objetivo.

Por otro lado, los parámetros que permanecen fijos a lo largo de todos los experimentos son:

- **Instancia:** p43.
- **Tamaño de la población:** 100.
- **Porcentaje de individuos generados aleatoriamente:** 0.8
- **Detenimiento del algoritmo en 1000 generaciones.**
- **Individuos seleccionados en el proceso de selección:** 10.
- **Operador de búsqueda local:** insertion.
- **Probabilidad de búsqueda local:** 0.8.
- **Operador de selección de sobrevivientes:** replace worst.
- **Individuos a reemplazar:** 10.

No todos los parámetros variables varían en todos los experimentos, sino que su variación se encuentra repartida a largo de los experimentos. La razón detrás de esto es que probar todas las combinaciones de todos los anteriores parámetros produce gráficos demasiado grandes y, en cierto punto, ilegibles. Por esta razón, los parámetros serán probados en grupos y, eventualmente, fijados para los experimentos posteriores.

La configuración correspondiente a cada experimento puede hallarse en el archivo de configuración **experimentN.yml** en **resources/configurations**. A su vez, sus resultados completos pueden hallarse en la carpeta **outputs/experimentN**. En ambos casos, **N** corresponde y debe reemplazarse por el número de experimento en cuestión.

Primer Experimento

El primer experimento consiste en la ejecución de 16 posibles configuraciones producto de la variación de los siguientes parámetros en los conjuntos de valores indicados:

- **Operador de cruce:** {OX1, PMX}.
- **Probabilidad de cruce:** {0.6, 0.9}.
- **Operador de mutación:** {swap, inversion}.
- **Probabilidad de mutación:** {0.1, 0.9}.

El número de ejecuciones por configuración se halla fijo en 30, mientras que el operador de selección se halla fijo en *tournament*, con $k = 3$. Esto, como se explicó, se realiza con la finalidad de mantener legibles los gráficos.

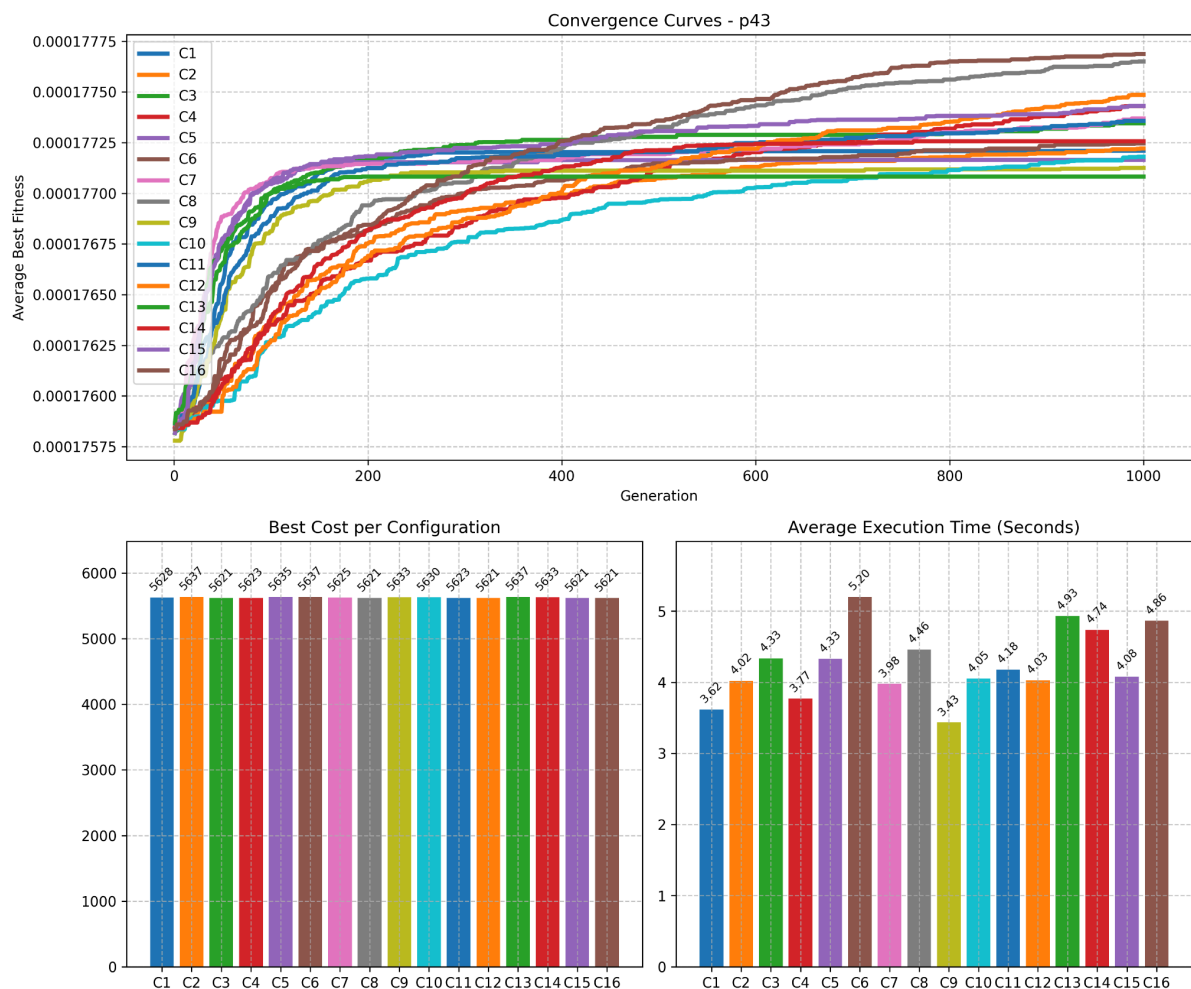


Imagen 2: Gráfico resultante de la ejecución de la aplicación con la configuración correspondiente al experimento 1.

Como puede observarse, las configuraciones que alcanzaron los mejores resultados para este experimento son las siguientes:

- C3: [OX1, 0.6, INVERSION, 0.1], 5621.
- C4: [OX1, 0.9, INVERSION, 0.9], 5623.
- C7: [OX1, 0.9, INVERSION, 0.1], 5625.
- C8: [OX1, 0.9, INVERSION, 0.9], 5621.
- C11: [PMX, 0.6, INVERSION, 0.1], 5623.
- C12: [PMX, 0.6, INVERSION, 0.9], 5621.
- C15: [PMX, 0.9, INVERSION, 0.1], 5621.
- C16: [PMX, 0.9, INVERSION, 0.9], 5621.

Considerando lo anterior, se realizan las siguientes observaciones:

- Los costos más bajos fueron todos conseguidos utilizando el operador de *inversion*. Esto indica que el operador de mutación pareciera tener el mayor peso a la hora de obtener mejores resultados, siendo este el parámetro común en todas las anteriores configuraciones.
- Las configuraciones con un *fitness* promedio más alto (C8 y C16) comparten una probabilidad de cruce y mutación alta. Esto indica que, en general, permiten obtener resultados más consistentes. No obstante, utilizar otras probabilidades igualmente permite alcanzar una buena solución. La distinción se halla en el desvío estándar de las soluciones y no necesariamente implica que las mayores probabilidades son mejores, sino que una alta probabilidad de cruce y mutación podrían estar provocando que las poblaciones encuentren óptimos locales de forma más sencilla.
- El operador de cruce pareciera no influir al momento de obtener una mejor solución.

Segundo Experimento

Se utiliza la misma configuración, con la excepción de que se reemplaza el operador de selección por *roulette*. Este cambio se realiza con la finalidad de determinar si el uso de *tournament* favoreció la victoria de *inversion* sobre *swap* o si esta fue independiente de este.

En la siguiente página es posible observar el gráfico resultante de este experimento. Las configuraciones C3, C4, C7, C8, C11, C12, C15 y C16 corresponden a la utilización de *inversion*, con las mismas descripciones brindadas en el experimento anterior.

En este caso, la diferencia no es tan clara y ambos algoritmos dieron resultados muy similares. No obstante, es posible notar algunos detalles con respecto a la prueba anterior:

- Las configuraciones que utilizan *inversion* con una probabilidad de mutación de 0.1 (C3, C7, C11 y C15) obtuvieron todas mejores resultados que aquellas que utilizan ese mismo operador pero con una probabilidad de mutación de 0.9 (C4, C8, C12 y C16). A su vez, el tiempo en el primer grupo fue menor al del segundo. Ambas diferencias probablemente se deben a la aplicación más frecuente del operador, puesto que esto se traduce en mayor costo computacional y, posiblemente, un aumento en el costo de los caminos al ser este más disruptivo.
- El operador PMX (C9 a C16), independientemente del operador de mutación, llegó en general a mejores valores que el operador OX1 (C1 a C8). No obstante, la diferencia pareciera no ser significativa.
- Al igual que en el experimento anterior, el menor costo se halló utilizando el operador de *inversion*.

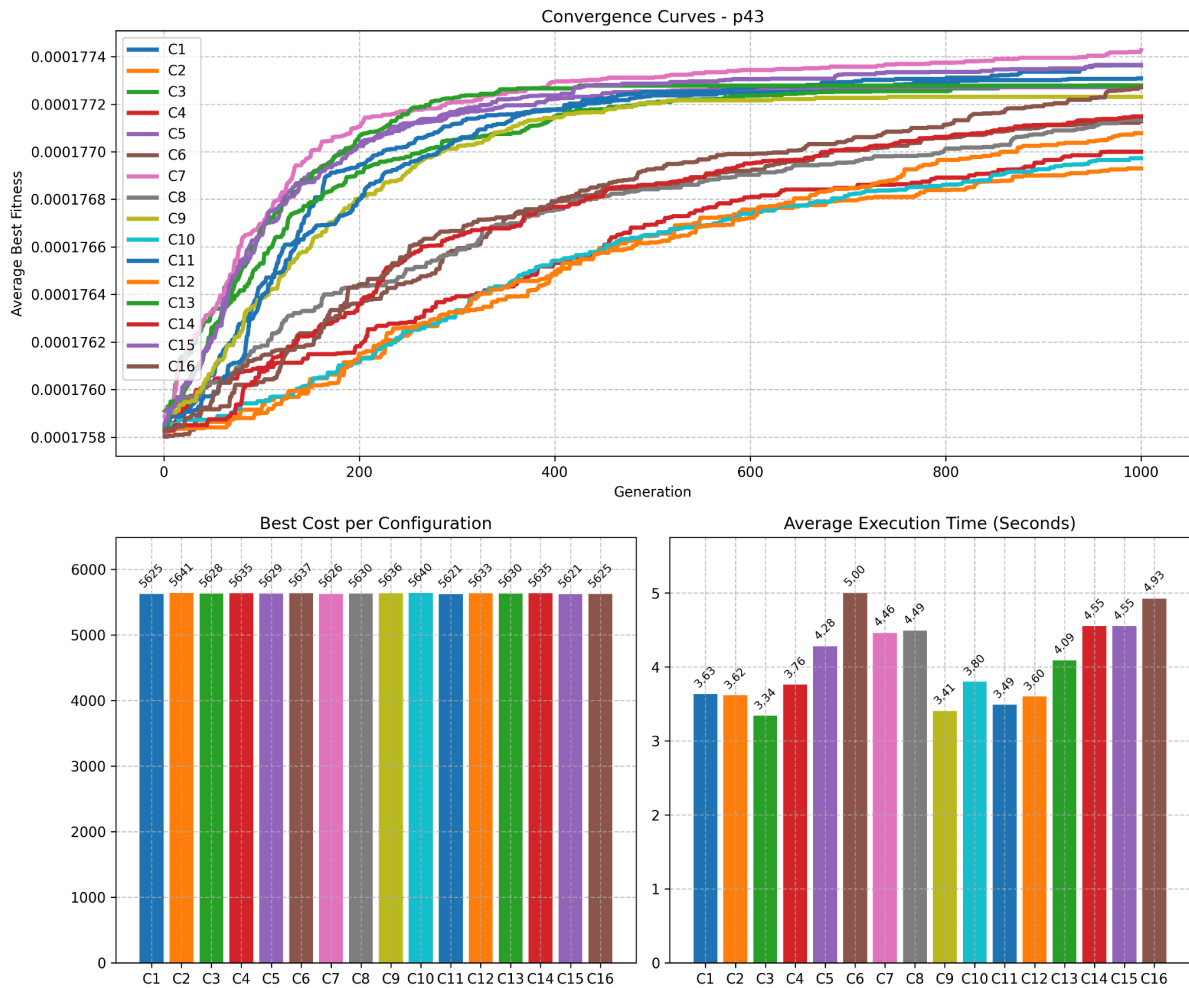


Imagen 3: Gráfico resultante de la ejecución de la aplicación con la configuración correspondiente al experimento 2.

En general, el operador de mutación *inversion*, pareciera, independientemente del operador de selección, comportarse mejor que el operador de *swap*, sobre todo para una probabilidad de mutación de 0.1. Adicionalmente, el operador *swap* con una probabilidad de 0.1 también demostró comportarse mejor, independientemente del operador de cruce y su probabilidad, que la configuración con estas mismas características, pero con una probabilidad de 0.9.

Debido a todo lo anterior, **se fija la probabilidad de mutación para los siguientes experimentos en 0.1.**

Tercer Experimento

En los anteriores dos experimentos, el operador de mutación *inversion* obtuvo mejores resultados con respecto a *swap*, independientemente del operador de selección utilizado, especialmente con una probabilidad de 0.1. En este experimento, se evaluará si el operador de *inversion* obtiene mejores resultados que el operador *scramble*, considerando ambos métodos de selección disponibles (*tournament*, con $k = 3$, y *roulette*) y una probabilidad de mutación fija de 0.1.

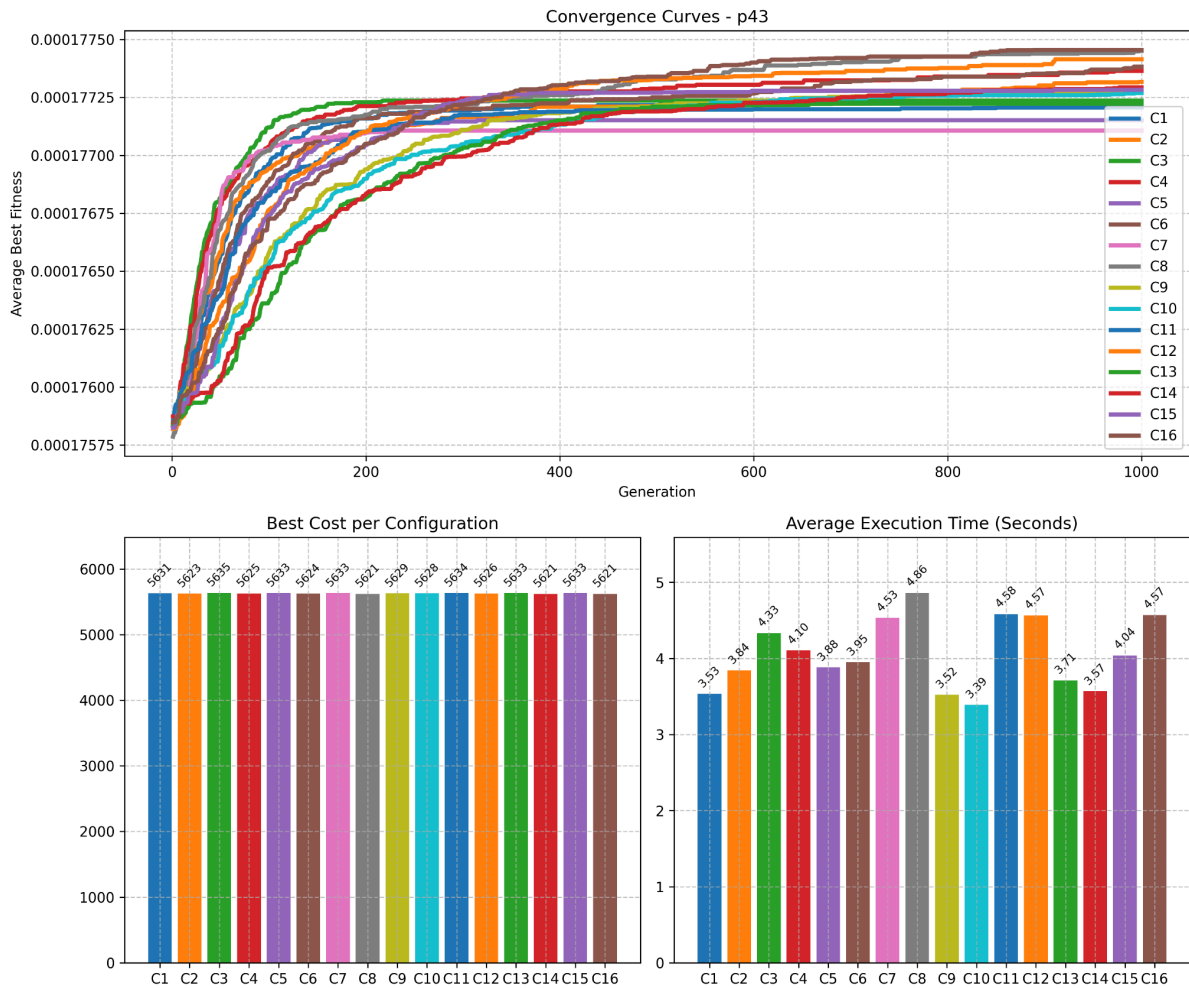


Imagen 4: Gráfico resultante de la ejecución de la aplicación con la configuración correspondiente al experimento 3.

Los mejores resultados fueron obtenidos por las configuraciones pares:

- C2: [TOURNAMENT, OX1, 0.6, INVERSION], 5623.
- C4: [TOURNAMENT, OX1, 0.9, INVERSION], 5625.
- C6: [TOURNAMENT, PMX, 0.6, INVERSION], 5624.
- C8: [TOURNAMENT, PMX, 0.9, INVERSION], 5621.
- C10: [ROULETTE, OX1, 0.6, INVERSION], 5628.
- C12: [ROULETTE, OX1, 0.9, INVERSION], 5626.
- C14: [ROULETTE, PMX, 0.6, INVERSION], 5621.
- C16: [ROULETTE, PMX, 0.9, INVERSION], 5621.

Nuevamente, estos mejores resultados se alcanzaron utilizando el operador *inversion*, quien anteriormente ya también los había obtenido contra el operador de mutación *swap*. Por consiguiente, parece lógico pensar que este, para la resolución de este problema y de esta instancia en particular, parece ser, de entre las opciones, el mejor operador de mutación. Por consiguiente, **se fija el operador de mutación en *inversion* para el resto de los experimentos**.

Con respecto a los operadores de selección, no se evidencian diferencias significativas.

En relación al operador de cruce, en los tres casos en los que se llegó al mejor valor obtenido (5621), fue con el operador PMX, en C8, C14 y C16. A pesar de que los resultados alcanzados por el operador OX1 son bastante cercanos (5625, 5624, 5628 y 5626), ninguno de ellos está igual de cerca al óptimo como lo están las configuraciones anteriormente mencionadas que utilizan PMX. Adicionalmente, en el anterior experimento, como se mencionó en las observaciones, PMX llegó también a mejores resultados generales que OX1. Por consiguiente, si se busca el menor costo, PMX pareciera ser la mejor opción. Por este motivo, **se fija PMX como operador de cruce**.

Cuarto Experimento

En este experimento, se utiliza la configuración descrita en el experimento anterior, con la diferencia de que, partiendo de las conclusiones anteriores, el operador de mutación se fija en *inversion* y el operador de cruce, en PMX.

Los parámetros que varían son:

- **Operador de selección:** [*tournament* ($k = 3$), *roulette*].
- **Probabilidad de cruce:** [0.6 y 0.9].

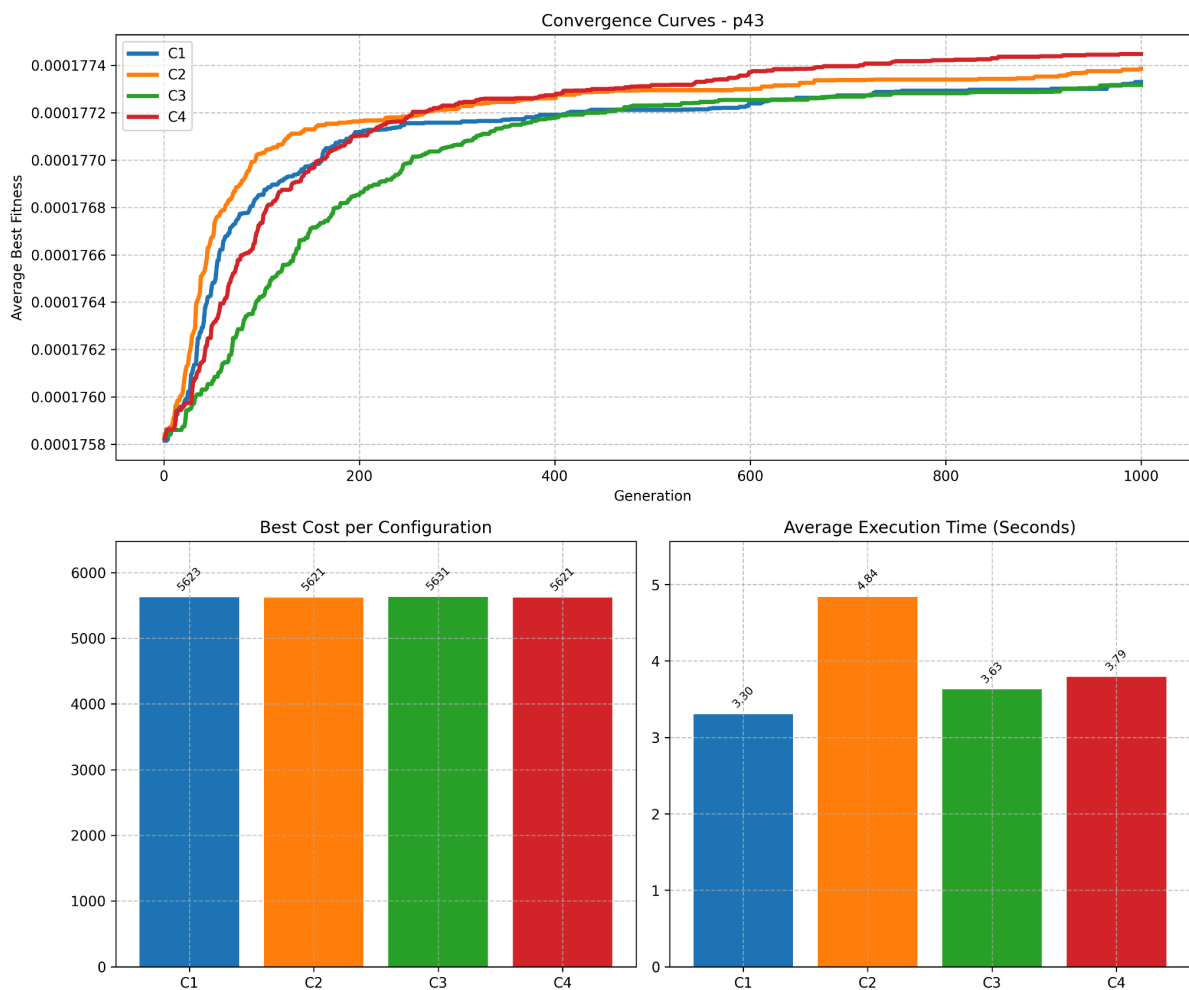


Imagen 5: Gráfico resultante de la ejecución de la aplicación con la configuración correspondiente al experimento 4.

Las configuraciones observables corresponden a:

- C1: [TOURNAMENT, $P(\text{crossover}) = 0.6$], 5623.
- C2: [TOURNAMENT, $P(\text{crossover}) = 0.9$], 5621.
- C3: [ROULETTE, $P(\text{crossover}) = 0.6$], 5631.
- C4: [ROULETTE, $P(\text{crossover}) = 0.9$], 5621.

Los mejores resultados fueron alcanzados por las configuraciones C2 y C4. Ambas comparten en común que utilizan una probabilidad de cruce de 0.9. Esto parece indicar que, para la instancia en particular, de las probabilidades probadas parece ser la mejor para resolver el problema. Por consiguiente, **se fija 0.9 como probabilidad de cruce para el resto de los experimentos.**

Con respecto al operador de selección, para una probabilidad de cruce de 0.9, ambos obtuvieron el mismo resultado. No obstante, si se observa el tiempo promedio, la ejecución de las configuraciones con el operador de *tournament* tardaron, en promedio, un segundo más. Por consiguiente, este pareciera, para dicha configuración, ser más lento que *roulette*.

Quinto Experimento

En este experimento, se busca determinar el mejor operador de selección. Para ello, se varía k entre 3, 5 y 8 y se aumenta el número de ejecuciones de 30 a 80, para una mayor precisión.

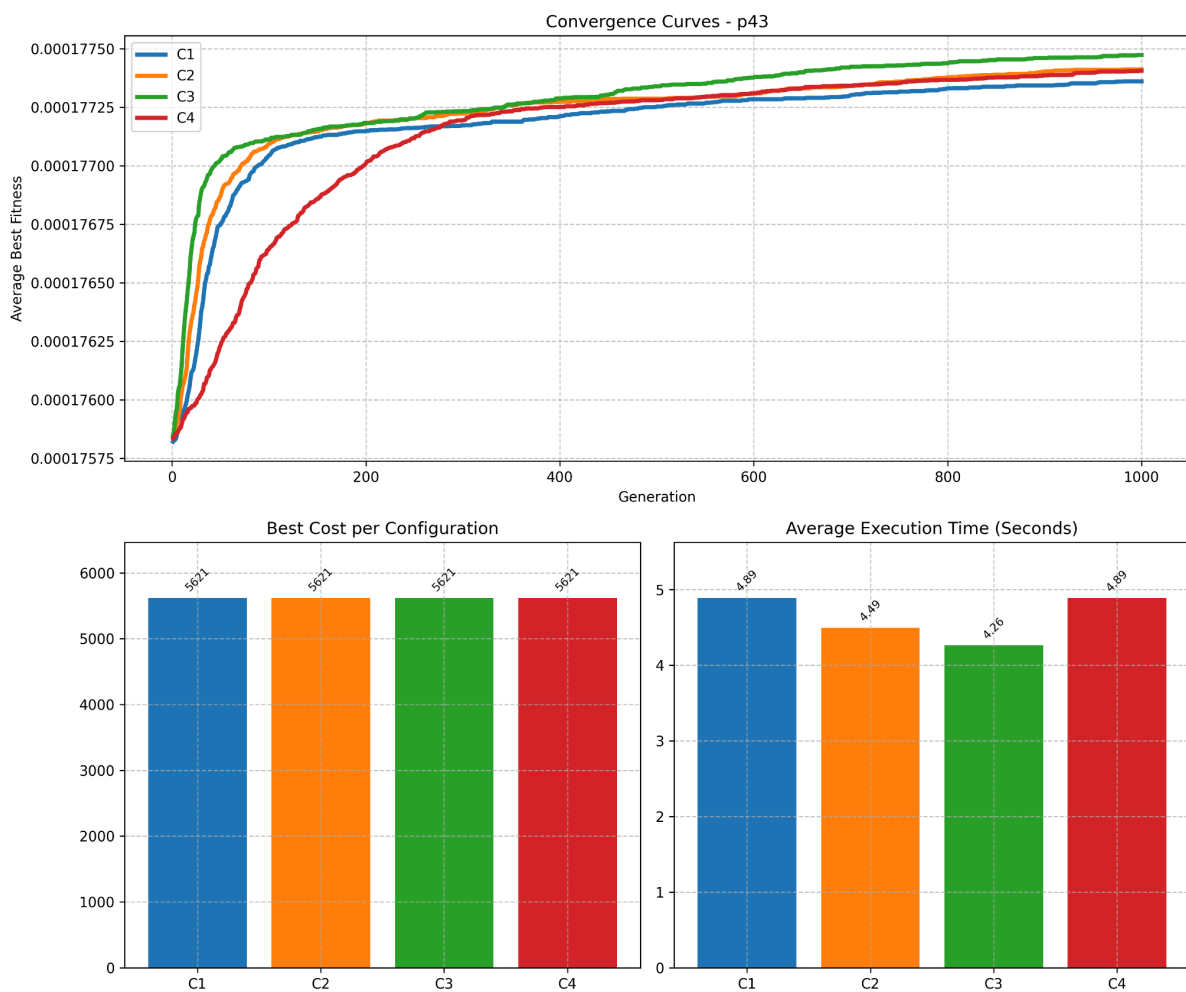


Imagen 6: Gráfico resultante de la ejecución de la aplicación con la configuración correspondiente al experimento 5.

La descripción del valor de los parámetros variables de cada instancia son los siguientes:

- C1: [TOURNAMENT, $k = 3$], 5621.
- C2: [TOURNAMENT, $k = 5$], 5621.
- C3: [TOURNAMENT, $k = 10$], 5621.
- C4: [ROULETTE], 5621.

Si bien las cuatro configuraciones alcanzan el mismo mejor valor (5621), es posible notar diferencias no solo en el inicio de las curvas, sino en los tiempos:

- *Roulette* en comparación con *tournament* produce, en promedio, peores resultados ante un menor número de generaciones. Es decir, si el número de generaciones es chico, *tournament* asegura alcanzar en promedio un mejor resultado que *roulette*.
- El parámetro k en *tournament* no solo asegura mejores soluciones iniciales, sino menores tiempos de ejecución promedios. Adicionalmente, a mayor es este parámetro (al menos para los valores testeados), mayor es la inclinación de la curva con respecto al eje x, lo que indica que a mayor es el k , menos generaciones se requieren para hallar un mejor valor.

Con todo lo descrito anteriormente, ***tournament* con $k = 10$ parece ser la mejor solución para la instancia “p43” del problema.**

Mejor Configuración

De acuerdo a los experimentos anteriores, la mejor configuración hallada para la instancia **p43** del problema, de las posibilidades que se probaron, es la siguiente:

- **Tamaño de la población:** 100.
- **Porcentaje de individuos generados aleatoriamente:** 0.8
- **Detenimiento del algoritmo en 1000 generaciones.**
- **Operador de selección:** *tournament*, con $k = 10$.
- **Individuos seleccionados en el proceso de selección:** 10.
- **Operador de cruce:** PMX.
- **Probabilidad de cruce:** 0.9.
- **Operador de mutación:** *inversion*.
- **Probabilidad de mutación:** 0.1.
- **Operador de búsqueda local:** *insertion*.
- **Probabilidad de búsqueda local:** 0.8.
- **Operador de selección de sobrevivientes:** *replace worst*.
- **Individuos a reemplazar:** 10.

Problemas Hallados

A continuación, se comparten algunos problemas hallados durante el desarrollo del algoritmo.

Endogamia

Durante el desarrollo de los experimentos, se encontró un problema: las poblaciones se llenaban de clones, sobre todo cuando el método de selección de padres era *tournament*. Esto generaba un estancamiento del algoritmo, sobre todo ante el uso de operadores poco disruptivos, como lo son *scramble* y *swap*.

Luego de un análisis del código, se halló la causa: ocurría que los individuos seleccionados algunas veces eran idénticos, lo que provocaba que, en el cruce, los hijos fuesen clones de ellos. Esto debido a que, si los padres son iguales, los operadores disponibles en el algoritmo, PMX y OX1, producen hijos idénticos.

La solución al problema fue que, **en caso de que los padres sean iguales, evitar su cruce**. Esto disminuyó el número de hijos en cada generación y, como consecuencia, también el número de reemplazos en el proceso de selección de supervivientes. No obstante, se evidenció un aumento en la diversidad de la población, lo que ayudó a evitar el estancamiento, facilitando la salida de óptimos locales y mejorando los resultados alcanzados por el algoritmo.

NumPy y Numba

Para optimizar el rendimiento de los distintos algoritmos, se evaluó la posibilidad de reemplazar las listas de listas actualmente utilizadas por arreglos de NumPy y, adicionalmente, utilizar Numba para compilar funciones muy utilizadas, como lo son el cálculo del *fitness*. No obstante, al llevar a cabo la implementación de dichos cambios, no se notó una mejora en el rendimiento, sino todo lo contrario: el tiempo de ejecución promedio para todo tipo de instancias, incluso para instancias simples como “*br17*”, incrementó de un poco menos de 2 segundos a 14 segundos (en ambos casos, con multiprocesamiento activo).

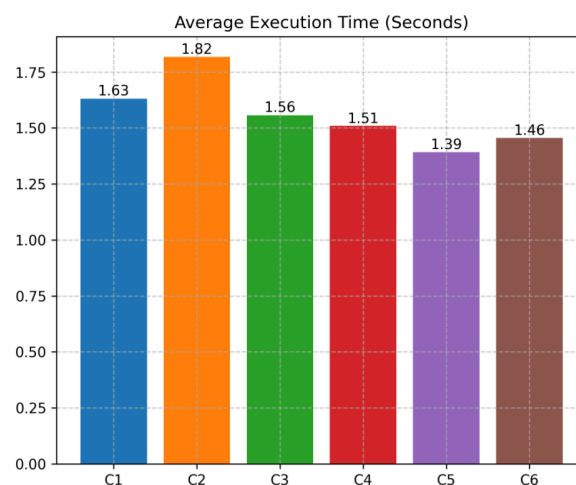


Imagen 7: Tiempo de ejecución promedio para seis configuraciones con la instancia “*br17*” previo a la aplicación de NumPy y Numba.

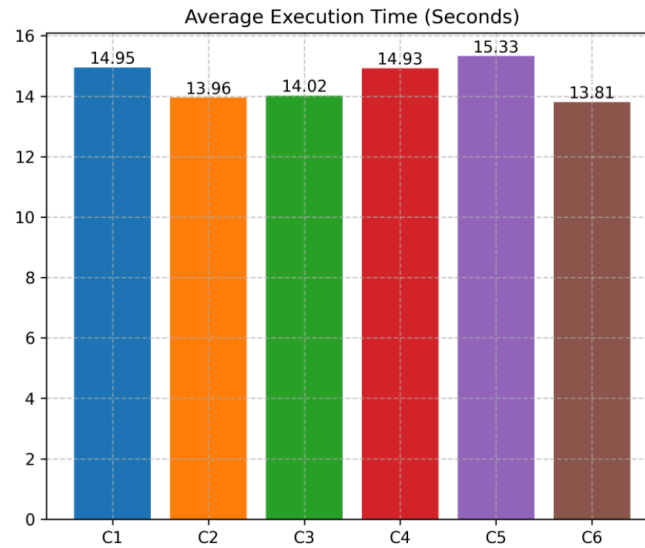


Imagen 8: Tiempo de ejecución promedio para seis configuraciones con la instancia “br17” posterior a la aplicación de NumPy y Numba.

No se encontró la causa por la que el uso de estructuras optimizadas producía el efecto contrario. Incluso a pesar de descartar Numba y utilizar únicamente NumPy con optimizaciones, los tiempos no mejoraron. Por consiguiente, se decidió descartar la idea.

Conclusiones

Las instancias del problema para las que se probó el algoritmo tienden a caer en óptimos locales de los que algoritmos de mutación como *swap* y *scramble* no permiten salir fácilmente. De acuerdo a los experimentos realizados, el operador de *inversion*, de entre los operadores de mutación disponibles en la aplicación, parece ser la mejor opción para alcanzar el resultado más cercano al óptimo posible (o incluso en algunas ocasiones llegando al óptimo). Por otro lado, la elección del algoritmo de selección de padres o de cruce parece no influir significativamente en el hallazgo de una mejor solución. No obstante, sí fue posible hallar algunas diferencias mínimas que podrían inclinar la preferencia hacia uno u otro.

Cabe destacar que los resultados obtenidos en este trabajo se encuentran altamente acoplados al diseño específico de la aplicación y a la implementación particular de cada uno de los algoritmos y operadores descritos. Asimismo, las conclusiones extraídas dependen estrechamente del conjunto de ejecuciones realizadas, por lo que variaciones en las semillas aleatorias o en el entorno de ejecución podrían alterar levemente los comportamientos observados.

Finalmente, es importante señalar que las comparaciones y valoraciones expuestas a lo largo de este informe se han realizado de forma intuitiva y visual a partir de las métricas capturadas. Si bien estas tendencias son claras en los gráficos presentados, para una validación formal y rigurosa en el ámbito científico sería necesario llevar a cabo un proceso estadístico (como pruebas de hipótesis). Dicho análisis permitiría evaluar con precisión matemática si las diferencias de rendimiento encontradas entre las distintas configuraciones son, o no, estadísticamente significativas.